

Guion de Exposicion

Patrones de Diseno

Proyecto: **Impacto Espacial** | Unreal Engine 4.27 | C++

Orden de exposicion (sigue el flujo del juego): **1) Singleton** → **2) Factory** → **3) Strategy** → **4) Observer**. El cerebro manda, la fabrica crea, cada enemigo se mueve y la interfaz reacciona.

Apertura (20-30 seg)

DI: "Mi juego es un shooter espacial lateral. Para organizarlo use 4 patrones de diseno. Les muestro primero el diagrama general y luego cada patron en el codigo."

MUESTRA: el diagrama UML con los 4 patrones. Senala los estereotipos («Singleton», «Factory», etc.).

1. Singleton - el cerebro del juego (UGameManager)

DI: "El Singleton garantiza que exista UNA sola instancia de una clase, accesible desde cualquier parte. Lo uso en el GameManager: lleva la puntuacion, el nivel y el estado del juego."

MUESTRA en **GameManager.h** y senala el mecanismo:

- El constructor es **privado** → nadie puede crear otro.
- El acceso es por un unico metodo **estatico** **ObtenerInstancia()** → siempre devuelve el mismo objeto.

FRASE CLAVE: "La nave, los enemigos y la UI consultan el mismo cerebro, sin duplicarlo."

2. Factory - quien crea los enemigos (UFabricaEnemigos)

DI: "La Fabrica centraliza la creacion de objetos. En vez de crear enemigos a mano por todos lados, le pido a la fabrica el tipo que quiero y me lo devuelve ya configurado."

MUESTRA en **FabricaEnemigos.cpp**: los metodos **GenerarEnemigo()** (switch por tipo) y **GenerarJefe()**.

- El generador no conoce las clases concretas: solo llama a **UFabricaEnemigos::GenerarJefe(...)** → **unico punto de creacion**.

FRASE CLAVE: "Si manana agrego un enemigo nuevo, solo toco la fabrica."

3. Strategy - como se mueve cada enemigo (UEstrategiaMovimiento)

DI: "Strategy permite tener varios algoritmos intercambiables. Cada enemigo tiene una estrategia de movimiento y se mueve segun ella, sin cambiar la clase Enemigo."

MUESTRA:

- La interfaz **UEstrategiaMovimiento** con el metodo **PURE_VIRTUAL** (contrato obligatorio).
- Las 4 hijas: Lineal, ZigZag, Diagonal, Perseguidor.
- En **EnemigoBase::Tick** delego: **EstrategiaActual->CalcularNuevaPosicion(...)**.

FRASE CLAVE: "El enemigo no sabe COMO se mueve; solo le pide a su estrategia. Puedo cambiarla hasta en pleno juego."

4. Observer - la interfaz reacciona sola (NaveJugador y PantallaHUD)

DI: "Observer permite que un objeto (el SUJETO) avise a otros (OBSERVADORES) cuando cambia, sin conocerlos. La nave es el sujeto y el HUD el observador."

MUESTRA el **publish** y el **subscribe** (lo que pide el docente):

- PUBLISH en **NaveJugador.cpp**: al cambiar la vida, **OnSaludCambiada.Broadcast(...)**.
- SUBSCRIBE en **PantallaHUD.cpp** (en **Observer**):
Nave->OnSaludCambiada.AddDynamic(this, &ActualizarSalud).

FRASE CLAVE: "La nave publica el cambio y el HUD se actualiza solo. La nave ni sabe que existe un HUD."

Cierre (15 seg)

DI: "En resumen: Singleton para el estado global, Factory para crear enemigos, Strategy para sus movimientos y Observer para la interfaz. Cada patron resuelve un problema concreto y mantiene el codigo ordenado y facil de ampliar."

Posibles preguntas del docente

Pregunta	Respuesta
Donde esta el publish del Observer?	En Broadcast() de la nave; la suscripcion es AddDynamic en el HUD.
Es un Singleton de verdad?	Si: constructor privado y acceso unico por ObtenerInstancia(). La instancia se guarda en el GameInstance para que sea segura entre niveles.
Tu Factory es Factory Method?	Es un Simple Factory (un metodo decide el tipo). Cumple lo esencial: el cliente no instancia clases concretas.

Como demuestras que Strategy es intercambiable?	Con EstablecerEstrategia() puedo cambiar el movimiento sin tocar la clase Enemigo.
---	--

Tip: ten 2 ventanas abiertas (el UML y el código) y por cada patrón salta del diagrama a la línea exacta. Eso demuestra que está implementado de verdad.

Diccionario de sintaxis (Unreal + C++)

Significado simple de las palabras que apareceran en tu codigo.

A) C++ basico

Palabra / sintaxis	Que significa
<code>class</code>	Define una clase: el molde con el que se crean los objetos.
<code>public / private / protected</code>	Quien puede usar un miembro: publico (todos), privado (solo la clase), protegido (la clase y sus hijas).
<code>void</code>	La funcion no devuelve ningun valor.
<code>virtual</code>	Metodo que las clases hijas PUEDEN sobrescribir (base del polimorfismo).
<code>virtual void Funcion()</code>	Funcion que no devuelve nada y que se puede sobrescribir en las hijas.
<code>override</code>	Indica que estas sobrescribiendo un metodo virtual del padre.
<code>PURE_VIRTUAL / = 0</code>	Metodo abstracto: la base no lo implementa; las hijas estan obligadas a hacerlo.
<code>static</code>	Pertenece a la CLASE, no a cada objeto. Se usa sin crear una instancia.
<code>: public AActor</code>	Herencia: esta clase hereda de AActor (es un tipo de AActor).
<code>* (puntero)</code>	Variable que guarda la direccion en memoria de un objeto.
<code>-></code>	Accede a un miembro a traves de un puntero (objeto->Metodo()).
<code>::</code>	Operador de ambito: separa clase y miembro (Clase::Metodo).
<code>&</code>	Referencia / 'direccion de'. En AddDynamic, &Clase::Funcion es 'la funcion'.
<code>const</code>	Marca algo que no se va a modificar.
<code>nullptr</code>	Puntero vacio: no apunta a nada.
<code>this</code>	Puntero al propio objeto que esta ejecutando el codigo.
<code>int32 / float / bool</code>	Tipos de dato: entero / decimal / verdadero-falso.
<code>switch</code>	Elige un camino segun el valor (como muchos if encadenados).

B) Unreal Engine (clases y funciones)

Palabra / sintaxis	Que significa
<code>UCLASS()</code>	Macro que registra la clase en Unreal (sistema de reflexion).
<code>GENERATED_BODY()</code>	Inserta el codigo que Unreal genera automaticamente para la clase.
<code>UPROPERTY()</code>	Marca una variable para que Unreal la gestione (recoleccion de basura, editor).
<code>UFUNCTION()</code>	Marca una funcion para Unreal. Necesaria para enlazarla a delegates/eventos.
<code>UObject</code>	Clase base de casi todo en Unreal.

Palabra / sintaxis	Que significa
AActor	Clase base de todo lo que se coloca en el mundo (tiene posicion).
APawn	Actor que un jugador o IA puede poseer y controlar (la nave).
UWorld	El mundo / nivel que se esta jugando ahora.
FVector	Vector 3D (X, Y, Z): sirve para posiciones y direcciones.
FRotator	Rotacion en grados (Pitch, Yaw, Roll).
TArray<>	Lista / arreglo dinamico de Unreal (la 'piscina' de balas, por ej.).
TSubclassOf<>	Referencia a una CLASE (no a un objeto); se asigna en el editor.
SpawnActor<>()	Crea (instancia) un actor dentro del mundo.
NewObject<>()	Crea un UObject que no es actor (ej. una estrategia).
Cast<>()	Convierte un puntero a otro tipo si es compatible; si no, da nullptr.
BeginPlay()	Se ejecuta una vez, al entrar el actor en juego.
EndPlay()	Se ejecuta al salir / destruirse el actor (para limpiar).
Tick(DeltaTime)	Se ejecuta cada frame. DeltaTime = segundos desde el frame anterior.
GetWorld()	Devuelve el mundo actual.
Destroy()	Elimina el actor del mundo.
FTimerHandle	Identificador de un temporizador.
SetTimer(...)	Programa que se llame una funcion tras X segundos (una o varias veces).
CreateDefaultSubobject<>()	Crea un componente del actor (solo en el constructor).
RootComponent	Componente raiz del actor: su base/ancla.
TakeDamage(...)	Funcion estandar de Unreal para recibir dano.
SetActorLocation(...)	Mueve el actor a una posicion.

C) Delegados / Observer (publish & subscribe)

Palabra / sintaxis	Que significa
DECLARE_DYNAMIC_MULTICAST_DELEGATE...	Declara un 'evento' al que se pueden suscribir varias funciones.
Broadcast(...)	DISPARA el evento: avisa a todos los suscritos. Es el 'publish' del Observer.
AddDynamic(obj, &Clase::Func)	SUSCRIBE una funcion al evento (el observador se engancha).
OnComponentBeginOverlap	Evento de Unreal: se dispara cuando dos cosas se solapan.
OnComponentHit	Evento de Unreal: se dispara cuando dos cosas chocan (bloqueo).
OnClicked	Evento de Unreal: se dispara al pulsar un boton de UI.